EXP 31. Prolog Program to Check Whether a Particular Bird Can Fly or Not

Aim: To determine whether a particular bird can fly or not using Prolog.

Program:

%%writefile bird.pl

```
bird(sparrow).
bird(parrot).
bird(pigeon).
bird(ostrich).
bird(penguin).

can_fly(sparrow).
can_fly(parrot).
can_fly(pigeon).

cannot_fly(Bird) :-
   bird(Bird),
   \+ can_fly(Bird).
```

Query and execution:

```
!swipl -q -s bird.pl -g "can_fly(sparrow), writeln('Sparrow can fly'), halt."
```

Output:



32. Prolog Program to Implement Family Tree

Aim: To implement family relationships such as mother, father, grandfather, grandmother, sister and brother.

Program:

The question gives genders but does not specify the parent-child connections. The following is a sample family tree to demonstrate the relations.

```
%%writefile family.pl

% Gender facts
female(pam).
female(liz).
female(ann).
female(pat).

male(tom).
male(bob).
male(jim).

% Sample parent relationships
parent(tom, liz).
parent(pam, liz).

parent(bob, ann).
parent(liz, ann).

parent(jim, pat).
parent(ann, pat).

% Mother relation
mother(X, Y) :-
   female(X),
```

```
    parent(X, Y).

% Father relation
father(X, Y) :-
    male(X),
    parent(X, Y).

% Grandfather relation
grandfather(X, Y) :-
    male(X),
    parent(X, Z),
    parent(Z, Y).

% Grandmother relation
grandmother(X, Y) :-
    female(X),
    parent(X, Z),
    parent(Z, Y).

% Sister relation
sister(X, Y) :-
    female(X),
    parent(Z, X),
    parent(Z, Y),
    X \= Y.

% Brother relation
brother(X, Y) :-
    male(X),
    parent(Z, X),
    parent(Z, Y),
```

X \= Y.

Queries and outputs:

!swipl -q -s family.pl -g "mother(pam, liz), writeln('Pam is mother of Liz'), halt."

OUTPUT:

```
% brother relation
brother(X, Y) :-
    male(X),
    parent(Z, X),
    parent(Z, Y),
    X \= Y.
```

```
Overwriting family.pl
```

```
!swipl -q -s family.pl -g "mother(pam, liz), writeln('Pam is mother of Liz'), halt."
```

```
Pam is mother of Liz
```

33. Prolog Program to Suggest Dieting System Based on Disease

Aim: To suggest a simple diet plan based on a disease.

Program:

%%writefile diet.pl

diet(diabetes, 'Balanced meals with controlled carbohydrates').

diet(hypertension, 'Low-salt diet with fruits and vegetables').

diet(obesity, 'Balanced calorie-controlled diet').

diet(anemia, 'Iron-rich foods such as beans and leafy greens').

suggest_diet(Disease, Diet) :-

diet(Disease, Diet).

Query:

!swipl -q -s diet.pl -g "suggest_diet(diabetes, D), write('Suggested Diet: '), writeln(D), halt."

OUTPUT:

```
    diet(Disease, Diet).
```

```
Writing diet.pl
```

```
!swipl -q -s diet.pl -g "suggest_diet(diabetes, D), write('Suggested Diet: '), writeln(D), halt."
```

```
Suggested Diet: Balanced meals with controlled carbohydrates
```

34. Prolog Program to Implement Monkey Banana Problem

Aim: To solve the Monkey Banana problem using Prolog.

Concept: The monkey wants to get bananas hanging from the ceiling. The monkey must move to the box, push it under the bananas, climb onto it and grasp the bananas.

Program:

```
%%writefile monkey.pl


% Initial state

initial_state(at(door), on(floor), at(window), has(no)).


% Goal

goal(has(banana)).


% Actions

move(at(X), at(Y)) :-

   X \= Y.


push_box(at(X), at(Y)) :-

   X \= Y.


climb(on(floor), on(box)).


grasp(has(no), has(banana)).


% Solution plan
```

```
solve([
    move(door, middle),
    push_box(middle, under_banana),
    climb(box),
    grasp(banana)
]).
```

Query:

```
!swipl -q -s monkey.pl -g "solve(Plan), writeln(Plan), halt."
```

Output:

```
    grasp(banana)
]).

Writing monkey.pl

!swipl -q -s monkey.pl -g "solve(Plan), writeln(Plan), halt."

[move(door,middle),push_box(middle,under_banana),climb(box),grasp(banana)]
```

35. Prolog Program for Fruits and Their Colors Using Backtracking

Aim: To display fruit names and their colors using Prolog backtracking.

Program:

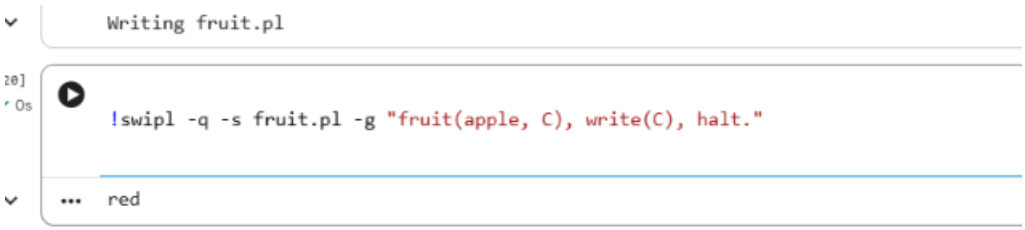
```
%%writefile fruit.pl

fruit(apple, red).
fruit(banana, yellow).
fruit(orange, orange).
fruit(grapes, green).
fruit(mango, yellow).
```

Query 1: Find the color of a fruit

!swipl -q -s fruit.pl -g "fruit(apple, C), write(C), halt."

OUTPUT



Query 2: Display all fruits and their colors using backtracking

!swipl -q -s fruit.pl -g "fruit(F, C), format('~w : ~w~n', [F,C]), fail."

OUTPUT:

